

LAPORAN TUGAS 5
“SISTEM BOOKING HOTEL”
SISTEM OPERASI

Dosen pengampu : Triawan Adi Cahyanto M.Kom



Disusun oleh :
NELSAN HADI SYAPUTRA
(2410651017)

UNIVERSITAS MUHAMMADIYAH JEMBER
FAKULTAS TEKNIK
PROGRAM STUDY TEKNIK INFORMATIKA

13-11-2025

1. Tujuan

- Menerapkan konsep **sinkronisasi proses** menggunakan **thread, semaphore, dan mutex** dalam pemrograman C.
- Mensimulasikan sistem **pemesanan kamar hotel** oleh banyak pelanggan secara bersamaan (multithreading).
- Menunjukkan bagaimana **sinkronisasi** mencegah *race condition* pada akses sumber daya bersama.

2. Dasar Teori

1. Thread

Thread adalah unit eksekusi terkecil dalam suatu proses. Dengan menggunakan thread, program dapat menjalankan beberapa tugas secara paralel.

2. Mutex (Mutual Exclusion)

Mutex digunakan untuk mengamankan *critical section* agar hanya satu thread yang dapat mengakses data bersama pada satu waktu.

3. Semaphore

Semaphore digunakan untuk membatasi jumlah thread yang dapat mengakses sumber daya terbatas.

- `sem_wait()` → menunggu jika resource penuh.
- `sem_post()` → memberi sinyal bahwa resource telah dilepas.

4. Sinkronisasi Proses

Sinkronisasi memastikan setiap thread bekerja secara bergantian dengan tertib tanpa menimbulkan konflik data.

3. Source Code Program

File: hotel_booking.c

Code source :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
```

```

#define TOTAL_ROOMS 5

#define TOTAL_CUSTOMERS 10

#define MAX_TRIES 3


sem_t rooms;          // semaphore kamar tersedia
pthread_mutex_t print_lock; // mutex untuk output rapi
int available_rooms = TOTAL_ROOMS; // variabel bantu untuk tampilan


void* customer(void* arg) {
    int id = *((int*)arg);
    int tries = 0;

    while (tries < MAX_TRIES) {
        tries++;

        pthread_mutex_lock(&print_lock);
        printf("[🕒] Customer-%d mencoba booking (percobaan ke-%d)...\n", id, tries);
        pthread_mutex_unlock(&print_lock);

        // mencoba booking (menunggu jika penuh)
        if (sem_trywait(&rooms) == 0) {
            pthread_mutex_lock(&print_lock);
            available_rooms--;

            printf("✅ Customer-%d BERHASIL booking kamar! (tersisa %d kamar)\n",
                id, available_rooms);
            pthread_mutex_unlock(&print_lock);

            sleep(3); // simulasi menginap

            pthread_mutex_lock(&print_lock);

```

```

    available_rooms++;

    printf("👤 Customer-%d check-out, kamar dikosongkan (tersisa %d kamar)\n",
           id, available_rooms);
    pthread_mutex_unlock(&print_lock);

    sem_post(&rooms); // lepas kamar
    pthread_exit(NULL);
} else {
    pthread_mutex_lock(&print_lock);
    printf("❌ Customer-%d gagal booking (kamar penuh), menunggu...\n", id);
    pthread_mutex_unlock(&print_lock);
    sleep(2);
}
}

pthread_mutex_lock(&print_lock);
printf("😞 Customer-%d menyerah setelah %d kali mencoba.\n", id, MAX_TRIES);
pthread_mutex_unlock(&print_lock);
pthread_exit(NULL);
}

int main() {
    srand(time(NULL));
    pthread_t customers[TOTAL_CUSTOMERS];
    int ids[TOTAL_CUSTOMERS];

    sem_init(&rooms, 0, TOTAL_ROOMS);
    pthread_mutex_init(&print_lock, NULL);

```

```
printf("=== 🏨 SISTEM BOOKING HOTEL ===\n");
printf("Total kamar tersedia: %d\n\n", TOTAL_ROOMS);

for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    ids[i] = i + 1;
    pthread_create(&customers[i], NULL, customer, &ids[i]);
    usleep(200000); // jeda agar output rapi
}

for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    pthread_join(customers[i], NULL);
}

printf("\n=== Semua customer selesai ===\n");

sem_destroy(&rooms);
pthread_mutex_destroy(&print_lock);
return 0;
}
```

4. Screenshot Output

Contoh tampilan output :

```
nelisan@NELSAN:~$ nano hotel_booking.c
nelisan@NELSAN:~$ gcc hotel_booking.c -o hotel_booking -lpthread
nelisan@NELSAN:~$ ./hotel_booking
=== [D] SISTEM BOOKING HOTEL ===
Total kamar tersedia: 5

[D] Customer-1 mencoba booking (percobaan ke-1)...
[D] Customer-1 BERHASIL booking kamar! (tersisa 4 kamar)
[D] Customer-2 mencoba booking (percobaan ke-1)...
[D] Customer-2 BERHASIL booking kamar! (tersisa 3 kamar)
[D] Customer-3 mencoba booking (percobaan ke-1)...
[D] Customer-3 BERHASIL booking kamar! (tersisa 2 kamar)
[D] Customer-4 mencoba booking (percobaan ke-1)...
[D] Customer-4 BERHASIL booking kamar! (tersisa 1 kamar)
[D] Customer-5 mencoba booking (percobaan ke-1)...
[D] Customer-5 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-6 mencoba booking (percobaan ke-1)...
[D] Customer-6 gagal booking (kamar penuh), menunggu...
[D] Customer-7 mencoba booking (percobaan ke-1)...
[D] Customer-7 gagal booking (kamar penuh), menunggu...
[D] Customer-8 mencoba booking (percobaan ke-1)...
[D] Customer-8 gagal booking (kamar penuh), menunggu...
[D] Customer-9 mencoba booking (percobaan ke-1)...
[D] Customer-9 gagal booking (kamar penuh), menunggu...
[D] Customer-10 mencoba booking (percobaan ke-1)...
[D] Customer-10 gagal booking (kamar penuh), menunggu...
[D] Customer-1 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-6 mencoba booking (percobaan ke-2)...
[D] Customer-6 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-2 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-7 mencoba booking (percobaan ke-2)...
[D] Customer-7 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-3 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-8 mencoba booking (percobaan ke-2)...
[D] Customer-8 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-4 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-9 mencoba booking (percobaan ke-2)...
[D] Customer-9 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-5 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-10 mencoba booking (percobaan ke-2)...
[D] Customer-10 BERHASIL booking kamar! (tersisa 0 kamar)
[D] Customer-6 check-out, kamar dikosongkan (tersisa 1 kamar)
[D] Customer-7 check-out, kamar dikosongkan (tersisa 2 kamar)
[D] Customer-8 check-out, kamar dikosongkan (tersisa 3 kamar)
[D] Customer-9 check-out, kamar dikosongkan (tersisa 4 kamar)
[D] Customer-10 check-out, kamar dikosongkan (tersisa 5 kamar)

=== Semua customer selesai ===
nelisan@NELSAN:~$
```

5. Penjelasan Program

- Program membuat **10 thread customer** yang dijalankan secara bersamaan.
- Setiap customer mencoba melakukan **booking kamar** menggunakan semaphore rooms.
- Jika kamar tersedia, customer dapat menginap dan kemudian melepaskan kamar setelah selesai.
- Jika kamar penuh, thread menunggu dan mencoba lagi hingga maksimal 3 kali.
- **Mutex** digunakan agar tampilan output antar-thread tidak saling bertumpuk.

6. Hasil dan Analisis

- Tanpa **sinkronisasi**, banyak thread dapat mengakses data kamar secara bersamaan dan menyebabkan *race condition*.
- Dengan **semaphore**, jumlah kamar yang bisa digunakan secara bersamaan dibatasi.
- Dengan **mutex**, program menjadi lebih teratur dan output mudah dibaca.
- Kombinasi kedua mekanisme ini menjaga sistem agar tetap **aman dan konsisten** walau berjalan paralel.

7. Cara Mengatasi Race Condition

- **Gunakan Mutex (Mutual Exclusion Lock)**

Mutex adalah mekanisme paling umum.
Hanya satu thread yang boleh masuk ke *critical section* (bagian kode yang mengakses data bersama).

- **Gunakan Semaphore**

Semaphore dapat membatasi **jumlah thread** yang boleh mengakses resource bersamaan.
Biasanya digunakan jika resource lebih dari satu (misal, 5 kamar hotel).

- **Gunakan Monitor / Condition Variable**

Pada bahasa seperti Java atau C++, *monitor* atau *condition variable* bisa digunakan agar thread menunggu dengan aman sampai kondisi tertentu terpenuhi.

- **Gunakan Atomic Operation**

Jika variabel yang diubah sederhana (misal counter), kamu bisa pakai **atomic variable** agar operasi read-modify-write dilakukan secara atomik oleh CPU.

- **Desain Program Tanpa Shared Data (Lock-Free Design)**

Kadang solusi terbaik adalah **menghindari data bersama sama sekali**:

- Gunakan pesan (*message passing*) antar thread.
- Gunakan *queue* yang aman terhadap thread (*thread-safe queue*).

8. Apa Yang Terjadi Jika Tidak Menggunakan Sinkronisasi

Kalau **tidak menggunakan sinkronisasi**, maka:

- Terjadi **race condition**, beberapa thread mengakses data bersamaan sehingga hasilnya salah.
- Data menjadi **tidak konsisten** (misal dua customer dapat kamar yang sama).
- **Output berantakan** karena thread mencetak secara bersamaan.
- Hasil program **tidak bisa diprediksi** dan bisa menyebabkan **crash atau deadlock**.

10. Kesimpulan

1. **Semaphore** berfungsi untuk mengatur jumlah sumber daya (kamar hotel).
2. **Mutex** melindungi proses agar hanya satu thread yang mengakses data bersama pada satu waktu.
3. Program berhasil menunjukkan konsep **sinkronisasi** pada sistem multi-threading.
4. Konsep ini bisa diterapkan untuk sistem nyata seperti **pemesanan hotel, parkir, atau sistem antrian**.